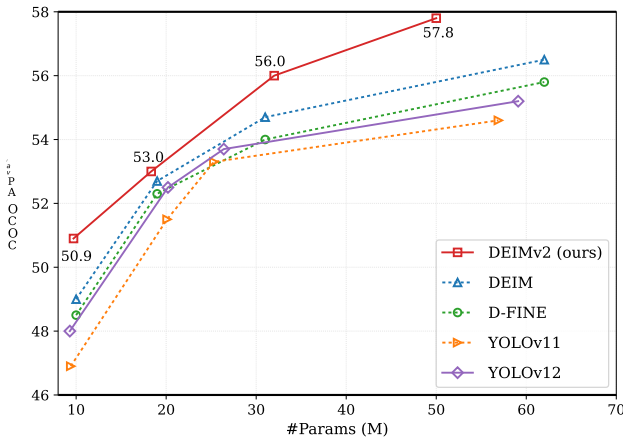


# 实时目标检测邂逅DINOv3

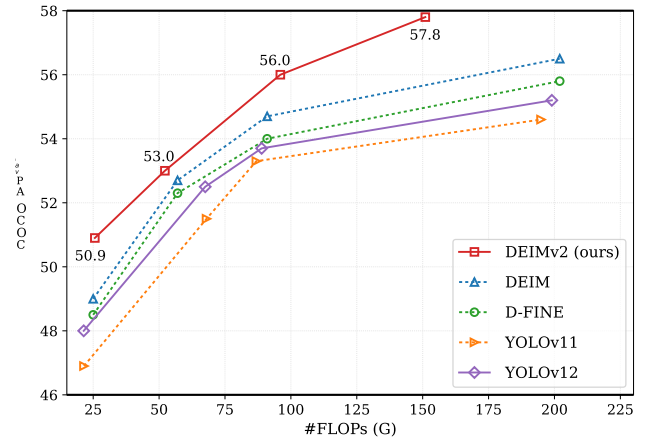
黄诗华<sup>1\*</sup>、侯永杰<sup>1,2\*</sup>、刘龙飞<sup>1\*</sup>、余轩龙<sup>1</sup>、沈曦<sup>1†</sup>

<sup>1</sup>Intellindust AI 实验室; <sup>2</sup>厦门大学 \* 同等贡献; <sup>†</sup> 通讯作者。

项目页面: <https://intellindust-ai-lab.github.io/projects/DEIMv2>  
代码与权重: <https://github.com/Intellindust-AI-Lab/DEIMv2>



(a) COCO performance v.s. Number of Parameters.



(b) COCO performance v.s. FLOPs.

图1. 与COCO数据集[12]上最先进的实时目标检测器相比, 我们提出的DEIMv2所有变体 (S、M、L、X) 在保持相似参数和更低计算成本的同时, 在平均精度 (AP) 方面实现了卓越性能。

## 摘要

Benefiting from the simplicity and effectiveness of Dense O2O and MAL, DEIM has become the mainstream training framework for real-time DETRs, significantly outperforming the YOLO series. In this work, we extend it with DINOv3 features, resulting in DEIMv2. DEIMv2 spans eight model sizes from X to Atto, covering GPU, edge, and mobile deployment. For the X, L, M, and S variants, we adopt DINOv3-pretrained / distilled backbones and introduce a Spatial Tuning Adapter (STA), which efficiently converts DINOv3's single-scale output into multi-scale features and complements strong semantics with fine-grained details to enhance detection. For ultra-lightweight models (Nano, Pico, Femto, and Atto), we employ HGNetv2 with depth and width pruning to meet strict resource budgets. Together with a simplified decoder and an upgraded Dense O2O, this unified design enables DEIMv2 to achieve a superior performance-cost trade-off across diverse scenarios, establishing new state-of-the-art results. Notably,

our largest model, DEIMv2-X, achieves 57.8 AP with only 50.3M parameters, surpassing prior X-scale models that require over 60M parameters for just 56.5 AP. On the compact side, DEIMv2-S is the first sub-10M model (9.71M) to exceed the 50 AP milestone on COCO, reaching 50.9 AP. Even the ultra-lightweight DEIMv2-Pico, with just 1.5M parameters, delivers 38.5 AP—matching YOLOv10-Nano (2.3M) with ~50% fewer parameters.

## 1. 引言

实时目标检测[6, 18, 22, 29]是许多实际应用中的关键组成部分, 包括自动驾驶[11]、机器人技术[16]和工业缺陷检测[8]。在检测性能与计算效率之间实现良好平衡仍然是一个关键挑战, 尤其对于适用于边缘和移动设备的轻量级模型而言。

在主流的实时目标检测器中, 基于DETR的方法因其

表1. DEIMv2变体的架构细节与主要结果。我们报告了不同变体的配置及其最终在COCO基准上的平均精度（AP）。al

| Variant | Backbone |         | Hidden Dimension   |                   |                   | Layers |       | #Scales | #Query | #Params | GFLOPs | Latency | AP   |
|---------|----------|---------|--------------------|-------------------|-------------------|--------|-------|---------|--------|---------|--------|---------|------|
|         | Model    | Adapter | d <sub>Back.</sub> | d <sub>Enc.</sub> | d <sub>Dec.</sub> | #Back. | #Dec. |         |        |         |        |         |      |
| X       | ViT-S+   | STA     | 384                | 256               | 256               | 12     | 6     | 3       | 300    | 50.26   | 151.6  | 13.75   | 57.8 |
| L       | ViT-S    | STA     | 384                | 256               | 256               | 12     | 4     | 3       | 300    | 32.18   | 96.32  | 10.47   | 56.0 |
| M       | ViT-T+   | STA     | 256                | 256               | 256               | 12     | 4     | 3       | 300    | 18.11   | 52.20  | 8.80    | 53.0 |
| S       | ViT-T    | STA     | 192                | 192               | 192               | 12     | 4     | 3       | 300    | 9.71    | 25.62  | 5.78    | 50.9 |
| Nano    | HGv2-B0  | -       | 1024               | 128               | 128               | 5      | 3     | 2       | 300    | 3.57    | 6.86   | 2.32    | 43.0 |
| Pico    | HGv2-P   | -       | 512                | 112               | 112               | 4      | 3     | 2       | 200    | 1.51    | 5.15   | 2.14    | 38.5 |
| Femto   | HGv2-F   | -       | 256                | 96                | 96                | 3      | 3     | 2       | 150    | 0.96    | 1.67   | 1.91    | 31.0 |
| Atto    | HGv2-A   | -       | 256                | 64                | 64                | 3      | 3     | 2       | 100    | 0.49    | 0.76   | 1.61    | 23.8 |

其端到端的设计以及Transformer带来的高容量，实现了更有利的权衡。在此范式下，DEIM已成为一个强大的训练框架，推动了实时DETR的发展，并在该领域提供了领先的模型。与此同时，DINOv3 [21] 已在多种视觉任务中展现出强大的特征表示能力。然而，它们在实时目标检测方面的潜力尚未得到充分探索。

在本工作中，我们推出了DEIMv2——一种基于先前DEIM[7]流程构建，并融合DINOv3[21]特征的实时目标检测器。DEIMv2在其最大规模变体（L和X尺寸）中采用官方DINOv3预训练主干网络（ViT-Small与ViT-Small+）以最大化特征丰富度；而S与M变体则采用从DINOv3蒸馏得到的ViT-Tiny与ViT-Tiny+主干网络，在性能与效率间实现精细平衡。针对超轻量级场景，我们进一步推出四种专用变体：Nano、Pico、Femto与Atto，将DEIMv2的可扩展性延伸至更广泛的计算资源范围。

为了更好地在实时约束下利用DINOv3在大规模数据上预训练所得的强大特征表示，我们设计了空间调谐适配器（STA）。STA与DINOv3并行运行，以无参数的方式高效地将其单尺度输出转换为目标检测所需的多尺度特征。同时，它对输入图像进行快速下采样，以提供具有极小感受野的细粒度多尺度细节特征，从而与DINOv3的强语义特征形成互补。

我们进一步借鉴Transformer领域的进展来简化解码器。具体而言，我们将传统的FFN和LayerNorm替换为SwishFFN[20]和RMSNorm[27]，这两者已被证明在效率上表现优异且对性能影响甚微。我们还注意到，在迭代优化过程中，对象查询位置的变化极小，这促使我们在所有解码器层之间共享查询位置嵌入。除此之外，我们通过引入对象级复制机制来增强Dense O2O—

混合增强，它增加了有效的监督并进一步提升了模型性能。

在COCO数据集[12]上进行的大量实验表明，DEIMv2在多种模型规模下均实现了最先进的性能，如图1所示。尽管设计简洁，DEIMv2系列模型展现出强大的性能。例如，我们最大的变体DEIMv2-X仅用5030万参数就在COCO上取得了57.6 AP，超越了先前最佳的X级检测器DEIM-X（后者需要超过6000万参数却仅获得56.5 AP）。在较小规模端，DEIMv2-S树立了一个显著里程碑，成为首个以不足1000万参数突破50 AP的模型，凸显了我们设计在紧凑尺度上的有效性。此外，我们超轻量级的DEIMv2-Pico仅用150万参数就达到了38.5 AP，其性能与YOLOv10-Nano（230万参数）相当，同时将参数量减少了~50%，从而在极端轻量化领域重新定义了效率与精度的边界。

我们的工作凸显了如何将DINOv3 [21]特征有效适配于实时目标检测，并提供了一个涵盖从超轻量级到高性能模型的通用框架。据我们所知，这是实时目标检测领域首个能同时应对如此广泛部署场景的研究。

本工作的主要贡献总结如下：

- 我们推出DEIMv2，提供涵盖GPU、边缘和移动部署的八种模型规模。
- 对于更大的模型，我们利用DINOv3获取强大的语义特征，并引入STA来高效地将这些特征集成到实时目标检测中。
- 对于超轻量级模型，我们利用专家知识有效剪枝HGNetv2-B0的深度和宽度，以满足严格的计算限制。
- 除了主干网络，我们进一步简化解码器并升级了Dense O2O，将性能边界推向更远。
- 最后，我们在COCO上证明DEIMv2优于

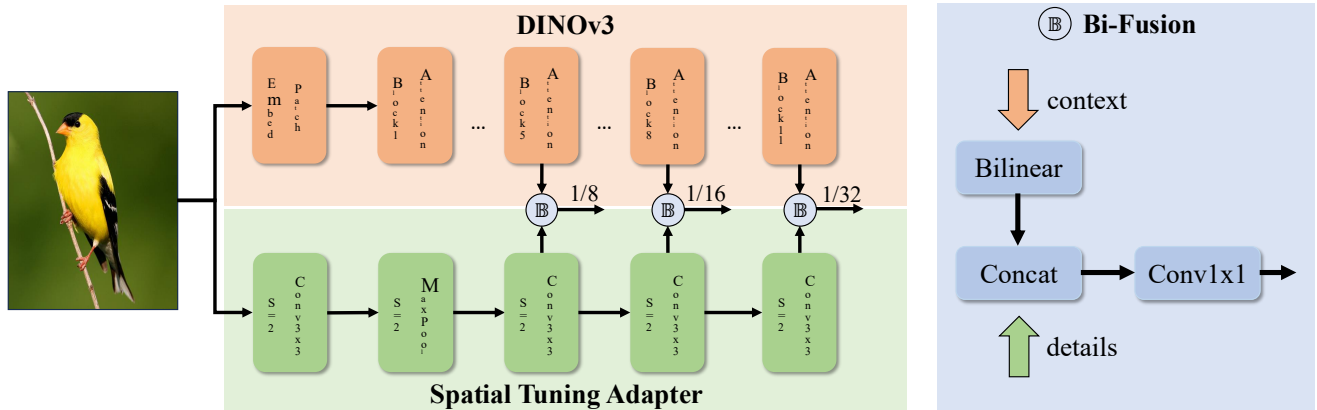


图2. 我们基于ViT变体的骨干网络设计。我们将DINOv3与提出的空间调谐适配器（STA）相结合。

在所有资源设置下形成了现有的最先进方法，确立了新的SOTA结果。

## 2. 方法

整体架构。我们的整体架构遵循RT-DETR[14]的设计，包含主干网络、混合编码器和解码器。如表1所示，对于主流的X、L、M和S变体，主干网络基于DINOv3并结合我们提出的空间调谐适配器（STA），其余变体则使用HGNetv2[1]。来自主干网络的多尺度特征首先由编码器处理，生成初始检测结果并筛选出前 $\{v^*\}$ 个候选边界框。解码器通过迭代优化这些候选框，最终生成预测结果。

基于ViT的变体。对于更大的DEIMv2变体（S、M、L、X），我们围绕视觉Transformer[3]（ViT）系列精心设计主干网络，平衡模型容量与效率。对于L和X，我们采用两个公开的DINOv3模型[21]：ViT-Small和ViT-Small+，它们通过12层网络和384维隐藏大小提供强大的语义表征。对于更轻量的S和M变体，我们直接从ViT-Small DINOv3蒸馏出紧凑主干网络ViT-Tiny和ViT-Tiny+，保持12层深度同时将隐藏维度分别降至192和256。这一设计在S  $\rightarrow$ 、M  $\rightarrow$ 、L  $\rightarrow$ 、X之间实现了平滑的扩展路径，确保每个变体在适应不同效率需求的同时保持竞争力的准确度。

基于HGNetv2的变体。由百度PaddlePaddle团队开发的HGNetv2 [1]因其高效性被广泛应用于实时DETR框架中——例如D-FINE [17]采用完整的HGNetv2系列作为其骨干网络。在我们超轻量级的DEIMv2模型（Nano，

在Pico、Femto和Atto系列中，我们同样基于HGNetv2-B0进行构建，但通过逐步剪裁其深度与宽度以适应不同的参数量级。具体而言，Pico主干网络移除了B0的第四阶段，仅保留至1/16尺度的输出特征。Femto在Pico基础上进一步将其末阶段的模块数量从两个缩减为一个。Atto则更进一步，将末层模块的通道数从512压缩至256。

空间调谐适配器。为了更好地使DINOv3特征适应实时目标检测任务，我们提出了空间调谐适配器（STA），其结构如图2所示。STA是一个全卷积网络，它集成了一个超轻量级前馈网络用于提取细粒度多尺度细节，同时配备双向融合算子以进一步增强DINOv3的特征表示能力。

DINOv3基于ViT骨干网络，天然生成单尺度（1/16）的密集特征。然而在目标检测中，物体尺寸差异巨大，多尺度特征正是提升性能最有效的手段之一。为此，ViTDet [10] 引入了Feature2Pyramid模块，通过反卷积从ViT最终输出中生成多尺度特征。相比之下，我们的STA方案更为简洁：我们直接通过无参数的双线性插值，将多个ViT块（例如第5、8、11层）的1/16尺度特征调整至多个尺度。这些多尺度特征会进一步通过双向融合算子增强——该算子包含 $1\times 1$ 卷积与专为提取细粒度细节设计的超轻量CNN，以补充DINOv3的输出特征。这种设计在效率与精度间取得了优异平衡，使其非常适合实时检测场景。

高效解码器。我们通过融入Transformer中广泛采用的几种效率导向技术，增强了标准可变形注意力解码器[31]。

表2. DEIMv2中详细的训练超参数。Back. 表示骨干网络。我们使用 $Local Loss$ 来表示D-FINE [17]中的细粒度定位（FGL）损失和解耦蒸馏焦点（DDF）损失。

| HyperParams              | X       | L       | M       | S        | Nano    | Pico     | Femto    | Atto     |
|--------------------------|---------|---------|---------|----------|---------|----------|----------|----------|
| Resolution               | 640     | 640     | 640     | 640      | 640     | 640      | 416      | 320      |
| Weight Decay             | 1.25e-4 | 1.25e-4 | 1e-4    | 1e-4     | 1e-4    | 1e-4     | 1e-4     | 1e-4     |
| Base LR                  | 5e-4    | 5e-4    | 5e-4    | 5e-4     | 8e-4    | 1.6e-3   | 1.6e-3   | 2e-3     |
| Min LR                   | 2.5e-4  | 2.5e-4  | 2.5e-4  | 2.5e-4   | 8e-4    | 8e-4     | 8e-4     | 1e-3     |
| Back. LR                 | 1e-6    | 1.25e-5 | 2.5e-5  | 2.5e-5   | 4e-4    | 8e-4     | 8e-4     | 1e-3     |
| Back. MinLR              | 5e-7    | 6.25e-6 | 1.25e-5 | 1.25e-5  | 4e-4    | 4e-4     | 4e-4     | 5e-4     |
| Local Loss               | ✓       | ✓       | ✓       | ✓        | ✓       |          |          |          |
| Epochs                   | 50      | 60      | 90      | 120      | 148     | 468      | 468      | 468      |
| Mosaic <sub>prob</sub>   | 0.5     | 0.5     | 0.5     | 0.5      | 0.5     | 0.5      | 0.5      | 0.3      |
| Mosaic <sub>epochs</sub> | [4, 29] | [4, 34] | [4, 49] | [4, 64]  | [4, 78] | [4, 250] | [4, 250] | [4, 250] |
| MixUp <sub>prob</sub>    | 0.5     | 0.5     | 0.5     | 0.5      | 0.5     | 0.0      | 0.0      | 0.0      |
| MixUp <sub>epochs</sub>  | [4, 29] | [4, 34] | [4, 49] | [4, 64]  | [4, 78] | []       | []       | []       |
| CopyB <sub>prob</sub>    | 0.5     | 0.5     | 0.5     | 0.5      | 0.4     | 0.0      | 0.0      | 0.0      |
| CopyB <sub>epoch</sub>   | [4, 50] | [4, 60] | [4, 90] | [4, 120] | [4, 78] | []       | []       | []       |

社区，实现了性能与成本之间的有利权衡。具体而言，我们集成了SwiGLUFFN[20]以增强非线性表示能力，采用RMSNorm[27]来高效稳定并加速训练。注意到对象查询位置在迭代优化过程中变化极小，我们进一步提出在所有解码器层之间共享单一位置嵌入，从而消除了冗余计算。

增强型密集O2O。在我们之前的DEIM [7]中，我们提出了密集O2O，它通过增加每张训练图像中的物体数量来提供更强的监督，从而改善收敛性和性能。其有效性最初通过使用Mosaic和MixUp [28]等图像级增强技术得到了验证。在DEIMv2中，我们进一步在物体层面探索了密集O2O，并引入了Copy-Blend方法，该方法能在不添加背景的情况下引入新物体。与完全覆盖目标区域的Copy-Paste [4]不同，Copy-Blend将新物体与图像进行融合，更适应我们的场景，并持续提升性能。

训练设置与损失函数。我们的训练策略遵循DEIM[7]，这是一个为快速收敛和高性能设计的基础框架。整体优化目标是五个组成部分的加权和：可匹配性感知损失（MAL）[7]、细粒度定位（FGL）损失[17]、解耦蒸馏焦点（DDF）损失[17]、边界框损失（L1）以及G IoU损失[19]。总损失定义为：

$$L_{total} = \lambda_1 L_{mal} + \lambda_2 L_{fgl} + \lambda_3 L_{ddf} + \lambda_4 L_{bbox} + \lambda_5 L_{giou} \quad (1)$$

权重在所有实验中设置为  $\lambda_1 = 1.0$ 、 $\lambda_2 = 0.15$ 、 $\lambda_3 = 1.5$ 、 $\lambda_4 = 5$  和  $\lambda_5 = 2$ 。

我们将训练超参数总结在表2中，涵盖输入分辨率、学习率、训练轮次以及Dense O2O设置。一个有趣的观察是，在超轻量级模型上应用FGL和DDF损失会导致性能下降。我们认为这是由于它们有限的能力和固有的较弱基线精度，这降低了自蒸馏的有效性。因此，我们在训练Pico、Femto和Atto变体时排除了这两个组件（即局部损失）。

### 3. 实验

与最先进的实时目标检测器对比。表3总结了DEIMv2在S、M、L和X变体上的性能表现，相较于先前的最先进检测器展现出显著提升。例如，最大变体DEIMv2-X仅以~50M参数和151 GFLOPs的计算量实现了57.8 AP，超越了先前最佳的DEIM-X（56.5 AP，62M参数和20.2 GFLOPs）。这表明DEIMv2能够以更少的参数和更低的计算成本提供更优的精度。在较小规模端，DEIMv2-S树立了新里程碑，成为首个在COCO数据集上以不足10M参数突破50 AP阈值的模型，仅以11M参数和26 GFLOPs即达到50.9 AP。这相较于先前的DEIM-S（49.0 AP，10M参数）有明显进步，同时所需模型规模几乎相同。尽管基于CNN的主干网络通常对硬件更友好，但我们基于ViT的主干网络以更少的参数实现了轻量化设计。

表3. 与COCO [12] val2017上实时目标检测器的比较，按参数量排序。

| Model                | #Epochs | #Params. | GFLOPs | Latency (ms) | $AP^{val}$ | $AP_{50}^{val}$ | $AP_{75}^{val}$ | $AP_S^{val}$ | $AP_M^{val}$ | $AP_L^{val}$ |
|----------------------|---------|----------|--------|--------------|------------|-----------------|-----------------|--------------|--------------|--------------|
| YOLOv10-S [23]       | 500     | 7        | 22     | 2.52         | 46.3       | 63.0            | 50.4            | 26.8         | 51.0         | 63.8         |
| YOLOv9-S [25]        | 500     | 7        | 26     | 8.05         | 46.8       | 61.8            | 48.6            | 25.7         | 49.9         | 61.0         |
| YOLOv12-S-turbo [22] | 600     | 9        | 19     | 6.28         | 47.5       | 64.1            | -               | -            | -            | -            |
| YOLO11-S [6]         | 500     | 9        | 22     | 7.05         | 46.6       | 63.4            | 50.3            | 28.7         | 51.3         | 64.1         |
| D-FINE-S [17]        | 120     | 10       | 25     | 3.66         | 48.5       | 65.6            | 52.6            | 29.1         | 52.2         | 65.4         |
| DEIM-S [7]           | 120     | 10       | 25     | 3.66         | 49.0       | 65.9            | 53.1            | 30.4         | 52.6         | 65.7         |
| YOLOv8-S [5]         | 500     | 11       | 29     | 6.97         | 44.9       | 61.8            | 48.6            | 25.7         | 49.9         | 61.0         |
| <b>DEIMv2-S</b>      | 120     | 10       | 26     | 5.78         | 50.9       | 68.3            | 55.1            | 31.4         | 55.3         | 70.3         |
| YOLOv10-M [23]       | 500     | 15       | 59     | 4.70         | 51.1       | 68.1            | 55.8            | 33.8         | 56.5         | 67.0         |
| D-FINE-M [17]        | 120     | 19       | 57     | 5.91         | 52.3       | 69.8            | 56.4            | 33.2         | 56.5         | 70.2         |
| DEIM-M [7]           | 90      | 19       | 57     | 5.91         | 52.7       | 70.0            | 57.3            | 35.3         | 56.7         | 69.5         |
| RT-DETRv2-S [14]     | 120     | 20       | 60     | 4.61         | 48.1       | 65.1            | 57.4            | 36.1         | 57.9         | 70.8         |
| YOLOv12-M-turbo [22] | 600     | 20       | 60     | 8.44         | 52.6       | 69.5            | -               | -            | -            | -            |
| YOLO11-M [6]         | 500     | 20       | 68     | 9.02         | 51.2       | 67.9            | 55.3            | 33.0         | 56.7         | 67.5         |
| YOLOv9-M [25]        | 500     | 20       | 76     | 10.18        | 51.4       | 67.2            | 54.6            | 32.0         | 55.7         | 66.4         |
| Gold-YOLO-S [24]     | 300     | 22       | 46     | 6.96         | 46.4       | 63.4            | -               | 25.3         | 51.3         | 63.6         |
| <b>DEIMv2-M</b>      | 90      | 18       | 52     | 8.80         | 53.0       | 70.2            | 57.6            | 34.2         | 57.4         | 71.5         |
| YOLOv10-L [23]       | 500     | 24       | 120    | 7.38         | 53.2       | 70.1            | 58.1            | 35.8         | 58.5         | 69.4         |
| YOLO11-L [6]         | 500     | 25       | 87     | 10.41        | 53.4       | 70.1            | 58.2            | 35.6         | 59.1         | 69.2         |
| YOLOv9-C [25]        | 500     | 25       | 102    | 10.76        | 53.0       | 70.2            | 57.8            | 36.2         | 58.5         | 69.3         |
| YOLOv8-M [5]         | 500     | 26       | 79     | 9.46         | 50.2       | 67.2            | 54.6            | 32.0         | 55.7         | 66.4         |
| YOLOv12-L-turbo [22] | 600     | 27       | 82     | 10.45        | 54.0       | 70.6            | -               | -            | -            | -            |
| YOLOv10-X [23]       | 500     | 30       | 160    | 10.47        | 54.4       | 71.3            | 59.3            | 37.0         | 59.8         | 70.9         |
| D-FINE-L [17]        | 72      | 31       | 91     | 8.15         | 54.0       | 71.6            | 58.4            | 36.5         | 58.0         | 71.9         |
| DEIM-L [7]           | 50      | 31       | 91     | 8.15         | 54.7       | 72.4            | 59.4            | 36.9         | 59.6         | 71.8         |
| RT-DETRv2-M [14]     | 120     | 31       | 92     | 6.91         | 49.9       | 67.5            | 58.6            | 35.8         | 58.6         | 72.1         |
| Gold-YOLO-M [24]     | 300     | 41       | 88     | 9.21         | 51.1       | 68.5            | -               | 32.3         | 56.1         | 68.6         |
| RT-DETRv2-L [14]     | 72      | 42       | 136    | 9.29         | 53.4       | 71.6            | 57.4            | 36.1         | 57.9         | 70.8         |
| YOLOv8-L [5]         | 500     | 43       | 165    | 12.20        | 52.9       | 69.8            | 57.5            | 35.3         | 58.3         | 69.8         |
| <b>DEIMv2-L</b>      | 60      | 32       | 96     | 10.47        | 56.0       | 73.4            | 60.9            | 37.5         | 60.8         | 75.2         |
| YOLOv9-E [25]        | 500     | 57       | 189    | 20.52        | 55.6       | 72.8            | 60.6            | 40.2         | 61.0         | 71.4         |
| YOLO11-X [6]         | 500     | 57       | 195    | 15.53        | 54.7       | 71.6            | 59.5            | 37.7         | 59.7         | 70.2         |
| YOLOv12-X-turbo [22] | 600     | 59       | 185    | 15.79        | 55.7       | 72.2            | -               | -            | -            | -            |
| D-FINE-X [17]        | 72      | 62       | 202    | 12.90        | 55.8       | 73.7            | 60.2            | 37.3         | 60.5         | 73.4         |
| DEIM-X [7]           | 50      | 62       | 202    | 12.90        | 56.5       | 74.0            | 61.5            | 38.8         | 61.4         | 74.2         |
| YOLOv8-X [5]         | 500     | 68       | 257    | 15.89        | 53.9       | 71.0            | 58.7            | 35.7         | 59.3         | 70.7         |
| Gold-YOLO-L [24]     | 300     | 75       | 152    | 12.31        | 53.3       | 70.9            | -               | 33.8         | 58.9         | 69.9         |
| RT-DETRv2-X [14]     | 72      | 76       | 259    | 13.88        | 54.3       | 72.8            | 58.8            | 35.8         | 58.8         | 72.1         |
| <b>DEIMv2-X</b>      | 50      | 50       | 151    | 13.75        | 57.8       | 75.4            | 63.2            | 39.2         | 62.9         | 75.9         |

参数和更低的浮点运算次数，提供了更好的可扩展性和部署灵活性。值得注意的是，所提出方法的延迟尚未优化。采用诸如Flash Attention [2]（如Yolov12 [22] 中所示）等技术可以进一步加速推理。总体而言，降低的浮点运算次数突显了基于ViT的主干网络在适当优化下实现低延迟性能的潜力。

有趣的是，在比较基于DINOv3的DEIMv2模型与之前的DEIM模型时，

在可比的参数量和FLOP预算下，准确率的提升主要来自中型和大型物体的改进，而小物体的性能基本保持不变。例如，DEIMv2-S达到了55.3  $AP_M$ 和70.3  $AP_L$ ，明显超过了DEIM-S（52.6  $AP_M$ 和65.7  $AP_L$ ），但小物体的得分几乎相同（31.4对比30.4  $AP_S$ ）。在更大的模型中也观察到了类似的趋势：DEIMv2-X将 $AP_M$ 从61.4提升至62.8， $AP_L$ 从74.2提升至75.9，而其小物体AP（39.2）则——

表4. 在COCO [12] val2017数据集上超轻量级模型与实时目标检测器的对比。

| Model                       | Input Size | Params (M) | FLOPs (G) | COCO AP     |
|-----------------------------|------------|------------|-----------|-------------|
| NanoDet-M[15]               | 416x416    | 1.0        | 0.7       | 23.5        |
| <b>DEIMv2-Atto</b>          | 320x320    | 0.5        | 0.8       | <b>23.8</b> |
| YOLOX-Nano[30]              | 416x416    | 0.9        | 1.1       | 25.8        |
| PicoDet-S[26]               | 416x416    | 1.0        | 1.2       | 30.7        |
| PP-YOLO-Tiny[13]            | 416x416    | 1.1        | 1.0       | 22.7        |
| PicoDet-ShufflenetV2 1x[26] | 416x416    | 1.2        | 1.5       | 30.0        |
| <b>DEIMv2-Femto</b>         | 416x416    | 1.0        | 1.7       | <b>31.0</b> |
| YOLOv10-N[23]               | 640x640    | 2.3        | 6.7       | 38.5        |
| YOLOv8-N[5]                 | 640x640    | 3.2        | 8.7       | 37.4        |
| YOLOv6-3.0-N[9]             | 640x640    | 4.7        | 11.4      | 37.0        |
| <b>DEIMv2-Pico</b>          | 640x640    | 1.5        | 5.2       | <b>38.5</b> |
| YOLOv12-N [22]              | 640x640    | 2.6        | 6.5       | 40.6        |
| DEIM-Nano [7]               | 640x640    | 3.8        | 7.2       | 43.0        |
| D-FINE-Nano [17]            | 640x640    | 3.8        | 7.2       | 42.8        |
| <b>DEIMv2-Nano</b>          | 640x640    | 3.6        | 6.9       | <b>43.0</b> |

接近DEIM-M（38.8）的主要指标。这些结果表明，DEIMv2的主要优势在于增强了对中大型物体的表征与检测能力，而小物体检测在不同尺度上仍具挑战性。这一观察进一步证实，DINOv3擅长捕捉强大的全局语义，但在表征细粒度细节方面能力有限。因此，探索如何将DINOv3特征更好地整合到实时检测器中，是未来工作中一个值得关注的方向。

与竞争性超轻量目标检测器的比较。  
DEIMv2的超轻量变体同样展现出强劲的性能，如表4所示。DEIMv2-Atto仅含0.49M参数，在模型尺寸大幅缩减的情况下，性能仍可与NanoDet-M相媲美。同样地，DEIMv2-Pico在参数数量不及一半的情况下，达到了与YOLOv10-N [23]相当的性能水平。这些结果印证了DEIMv2在极端紧凑模型架构中的有效性，并凸显了其在资源受限边缘设备上的部署优势。

4. 结论

在本报告中，我们推出了DEIMv2——新一代实时目标检测器，它将DINOv3强大的语义表征能力与我们轻量化的STA模块相结合。通过精心设计与规模调整，DEIMv2在全系列模型尺寸上均实现了最先进的性能表现。在高端配置中，DEIMv2-X以57.8 AP的精度显著减少了参数量，优于先前的大型检测器；在紧凑端，DEIMv2-S成为同尺寸首款突破50 AP的模型，其超

轻量级DEIMv2-Pico在参数量减少超过50%的情况下，性能与YOLOv10-N相当。这些结果共同表明，DEIMv2不仅高效，而且具备高度可扩展性，提供了一个推动精度-效率边界前进的统一框架。这种多功能性使DEIMv2非常适合部署于多样化场景——从资源受限的边缘设备到高性能检测系统，为实际应用中更广泛地采用实时检测技术铺平了道路。

参考文献

[1] 程翠、郭若愚、杜宇宁、贺栋梁、李甫、吴泽武、刘奇文、温世磊、黄继周、胡晓光等。超越自监督：一种简单而有效的网络蒸馏替代方案以改进骨干网络。arXiv, 2021年。3[2] Tri Dao、Dan Fu、Stefano Ermon、Atri Rudra 和 Christopher Ré。Flashattention：具有IO感知能力的快速且内存高效的精确注意力机制。发表于NeurIPS，2022年。5 [3] Alexey Dosovitskiy、Lucas Beyer、Alexander Kolesnikov、Dirk Weissenborn、翟晓华、Thomas Unterthiner、Mostafa Dehghani、Matthias Minderer、Georg Heigold、Sylvain Gelly等。一幅图像值16x16个词：大规模图像识别的Transformer模型。发表于ICLR，2020年。3[4] Golnaz Ghiasi、Yin Cui、Aravind Srinivas、钱瑞、林宗仪、Ekin D Cubuk、Quoc V Le 和 Barret Zoph。简单的复制粘贴是实例分割的一种强大数据增强方法。发表于CVPR，2021年。4[5] Jocher Glenn。Yolov8。https://docs.ultralytics.com/models/yolov8/，2023年。5, 6[6] Jocher Glenn。Yolo11。https://docs.ultralytics.com/models/yolo11/，2024年。1, 5

[7] 黄世华, 陆志超, 蔡晓东, 于永军, 周晓, 沈曦. DEIM: 具有改进匹配机制的DETR, 用于快速收敛。于*CVPR*, 2025年。2, 4, 5, 6[8] Muhammad Hussain. 从YOLO-v1到YOLO-v8: YOLO的崛起及其对数字化制造和工业缺陷检测的互补性。2023年。1[9] 李楚怡, 李露露, 耿一飞, 蒋洪亮, 程猛, 张博, 柯在丹, 徐晓明, 褚祥祥. YOLOv6 v3.0: 全面重装。arXiv, 2023年。6[10] 李阳浩, 毛汉子, Ross Girshick, 何恺明. 探索用于目标检测的朴素视觉Transformer骨干网络。于*ECCV*, 2022年。3[11] 梁思远, 吴昊, 甄丽, 华乔智, Sahil Garg, Georges Kaddoum, Mohammad Mehedi Hassan, 于科平. Edge YOLO: 基于自动驾驶车辆边缘-云协同的实时智能目标检测系统。*IEEE Transactions on Intelligent Transportation Systems*, 2022年。1[12] 林腾毅, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, C Lawrence Zitnick. Microsoft COCO: 上下文中的常见物体。于*ECCV*, 2014年。1, 2, 5, 6[13] 龙翔, 邓凯鹏, 王冠中, 张阳, 党青青, 高原, 沈辉, 任建国, 韩淑敏, 丁二锐, 闻世磊. PP-YOLO: 一种有效且高效的目标检测器实现。arXiv, 2020年。6[14] 吕文宇, 赵一安, 常钦尧, 黄奎, 王冠中, 刘毅. RT-DETRv2: 通过免费技巧包改进的实时检测Transformer基线。arXiv, 2024年。3, 5[15] Rangi Lyu. NanoDet-Plus. <https://github.com/RangiLyu/nanodet/releases/tag/v1.0.0-alpha-1>, 2021年。版本 1.0.0-alpha-1。6[16] Debapriya Maji, Soyeb Nagori, Manu Mathew, Deepak Poddar. YOLO-6D-Pose: 增强YOLO用于单阶段单目多物体6D姿态估计。于*3DV*, 2024年。1[17] 彭岩松, 李河北, 吴培熙, 张悦怡, 孙晓燕, 吴枫. D-Fine: 将DETR中的回归任务重新定义为细粒度分布优化。于*ICLR*, 2024年。3, 4, 5, 6[18] Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi. 你只看一次: 统一的实时目标检测。于*CVPR*, 2016年。1[19] Hamid Rezaatoughi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, Silvio Savarese. 广义交并比: 一种边界框回归的度量与损失函数。于*CVPR*, 2019年。4[20] Noam Shazeer. GLU变体改进Transformer。arXiv preprint arXiv:2002.05202, 2020年。2, 4[21] Oriane Siméoni, Huy V Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, 等. DinoV3。arXiv, 2025年。2, 3[22] 田云杰, 叶启祥, David Doermann. YOLOv12: 以注意力为中心的实时目标检测器。于*NeurIPS*, 2025年。1, 5, 6

[23] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, and Guiguang Ding. Yolov10: 实时端到端目标检测。于*NeurIPS*, 2024。5, 6[24] Chengcheng Wang, Wei He, Ying Nie, Jianyuan Guo, Chuanjian Liu, Yunhe Wang, and Kai Han. Gold-yolo: 基于收集-分发机制的高效目标检测器。于*NeurIPS*, 2023。5[25] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. Yolov9: 使用可编程梯度信息学习所需内容。arXiv, 2024。5[26] Guanghua Yu, Qinyao Chang, Wenyu Lv, Chang Xu, Cheng Cui, Wei Ji, Qingqing Dang, Kaipeng Deng, Guanzhong Wang, Yuning Du, et al. Pp-picodet: 移动设备上更优的实时目标检测器。arXiv, 2021。6[27] Biao Zhang and Rico Sennrich. 均方根层归一化。于*NeurIPS*, 2019。2, 4[28] Hongyi Zhang. mixup: 超越经验风险最小化。于*ICLR*, 2017。4[29] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. 在实时目标检测中DETR超越YOLO。于*CVPR*, 2024。1[30] Ge Zheng, Liu Songtao, Wang Feng, Li Zeming, and Sun Jian. YOLOx: 2021年超越YOLO系列。arXiv, 2021。6[31] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogan Wang, and Jifeng Dai. 可变形DETR: 用于端到端目标检测的可变形Transformer。于*ICLR*, 2021。3